

课程笔记

课程笔记：Lecture 6A 强化学习 (Reinforcement Learning)

Codex

2026 年 4 月 16 日

Robots that Learn with **Reinforcement Learning**

CS 294-277 Guest Lecture

Haozhi Qi

1

视频作者/频道：Robots That Learn / UC Berkeley CS 294-277

发布日期：课程讲次日期：2024-10-14

视频时长：47:38

视频链接：<https://www.youtube.com/watch?v=PRMjicOL0rk>

目录

1 课程定位与本讲学习目标	2
1.1 本讲想解决什么问题	2
1.2 课程摘录与结构对齐	2
1.3 本章小结	3
2 从 MDP 到策略优化	3
2.1 Markov Decision Process	3
2.2 价值函数、动作价值与优势函数	3
2.3 RL 的三维分类	4
2.4 本章小结	4
3 Policy Gradient 线路：REINFORCE、Actor-Critic 与 PPO	4
3.1 REINFORCE：从轨迹中学策略	4
3.2 REINFORCE 的问题：高方差	4
3.3 Actor-Critic：用 critic 降低方差	5
3.4 PPO：限制每次更新的幅度	5
3.5 本章小结	5
4 把 RL 用到机器人：现实中的优势与困难	6
4.1 为什么机器人是 RL 的自然应用场景	6
4.2 真实世界 RL 的四个困难	6
4.3 sim-to-real 作为折中路径	6
4.4 本章小结	6
5 模拟器如何工作：从动力学到碰撞检测	6
5.1 为什么需要模拟器	6
5.2 模拟循环的三部分	7
5.3 牛顿与拉格朗日视角	7
5.4 本章小结	7
6 总结与延伸	7
6.1 延伸阅读	7
6.2 最后一句话	7

Robots that Learn with Reinforcement Learning

CS 294-277 Guest Lecture

Haozhi Qi

1

图 1: 课程官方材料中的代表性页面, 用于帮助读者在进入本讲正文前建立整体上下文。

1 课程定位与本讲学习目标

这一讲是整门课的强化学习入门与机器人应用导论。讲者先把 RL 的基本对象、常见算法族和它在机器人里的意义讲清楚, 再把视角推进到两个现实问题: 为什么机器人 RL 必须依赖高质量模拟器, 以及为什么 sim-to-real (simulation-to-real) 不是可选项, 而是几乎所有机器人 RL 方案都绕不开的核心难题。

1.1 本讲想解决什么问题

从教学顺序看, 这一讲不是“再讲一次 RL 教科书”, 而是回答三个实践问题:

- RL 到底在优化什么, 和监督学习有什么本质差异;
- 机器人为什么会天然吸引 RL, 但同时又最难落地;
- 当我们真的要训练机器人策略时, simulator、parallel simulation 与 sim-to-real gap 分别扮演什么角色。

本讲的中心立场

强化学习在机器人里最有价值的地方, 不是“它很酷”, 而是它提供了一个能把长期回报、物理交互和策略优化统一起来的框架; 但这个框架能否落地, 极度依赖模拟器、训练稳定性和现实转移能力。

1.2 课程摘录与结构对齐

课程摘录把这一讲分成两大半: 前半部分回顾 MDP、REINFORCE、actor-critic 与 PPO, 后半部分讨论机器人应用、真实世界 RL 的局限、模拟器的物理循环, 以及 sim-to-real 的基本思路。正文也沿着这个顺序展开。

1.3 本章小结

本讲的重点不是记住某一个算法，而是建立一张“RL 在机器人里到底怎样工作”的地图：先看问题形式，再看算法族，再看真实部署时为什么必须依赖模拟和转移技巧。

2 从 MDP 到策略优化

2.1 Markov Decision Process

强化学习通常被形式化为 Markov Decision Process (MDP, 马尔可夫决策过程)。一个标准 MDP 可写成

$$(\mathcal{S}, \mathcal{A}, R, T, \gamma),$$

其中：

- $\mathcal{S}$ 是状态空间 (state space);
- $\mathcal{A}$ 是动作空间 (action space);
- $R: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 是奖励函数 (reward function);
- $T: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}_+$ 是状态转移;
- $\gamma \in [0, 1]$ 是折扣因子 (discount factor)。

对应的优化目标是寻找策略 π 使期望累计回报最大：

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t R_t(s_t, a_t, s_{t+1}) \mid \pi \right].$$

这里的策略 $\pi(a \mid s)$ 可以理解为“在状态 s 下选择动作 a 的概率分布”。与监督学习不同，策略的输出不是静态标签，而是会改变后续状态分布的行为本身。

2.2 价值函数、动作价值与优势函数

为了把长期回报拆开分析，课程引入三类最常用的函数：

$$Q_{\pi}(s_t, a_t) = \mathbb{E}_{\pi} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \mid s_t, a_t \right],$$

$$V_{\pi}(s_t) = \mathbb{E}_{a \sim \pi(\cdot \mid s_t)} [Q_{\pi}(s_t, a)],$$

$$A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t).$$

它们分别表示：

- Q_{π} ：在给定状态和动作后，后续还能拿到多少总回报；
- V_{π} ：在该状态下，按照当前策略平均能得到多好的回报；
- A_{π} ：某个动作比“在这个状态下的平均水平”好多少。

为什么要引入优势函数 (advantage function)

只用回报做策略梯度，方差通常很大。优势函数相当于把“这个动作到底好不好”与“这个状态本来就好不好”分开，从而减少噪声、稳定训练。

2.3 RL 的三维分类

课程把 RL 的主要分歧整理成三组坐标：

- model-based vs. model-free：是否显式学习或使用环境动力学模型；
- value-based vs. policy-based：是先估值再导出策略，还是直接学策略；
- on-policy vs. off-policy：训练数据是否必须来自当前策略。

这三组坐标不是互相独立的装饰，而是决定算法性质的核心轴。比如 actor-critic 同时用策略网络和价值网络；PPO 是典型的 on-policy 方法；而 model-based 方法通常更样本高效，但工程复杂度也更高。

为什么算法 taxonomy 重要

如果不先分清自己是在学模型、学价值还是学策略，就很难判断一个方法为什么会稳定、为什么会省样本、为什么会在真实机器人上失败。

2.4 本章小结

MDP 给出了 RL 的数学语言，value / policy / advantage 给出了最常见的分解方式，而三维 taxonomy 则帮我们判断一个方法到底属于哪一类、适不适合机器人任务。

3 Policy Gradient 线路：REINFORCE、Actor-Critic 与 PPO

3.1 REINFORCE：从轨迹中学策略

REINFORCE 的关键出发点很朴素：在很多 RL 问题里，并不存在监督学习意义上的 ground-truth action。比如 Pong 里，你看见的是像素，真正需要学的是“该往上还是往下”，而训练信号往往只在整条轨迹结束后才以胜负形式出现。

把一条轨迹记为 τ ，目标可以写成

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right].$$

其梯度的蒙特卡洛估计可写为

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_i \sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) r(\tau_i).$$

这条公式的直觉很重要：如果某条轨迹结果好，那么让导致它的动作更容易被采样；如果结果差，就把这些动作的概率压低。它看起来很像最大似然训练，只不过标签不是人工给的，而是环境反馈。

3.2 REINFORCE 的问题：高方差

REINFORCE 的缺点也非常直接：只靠轨迹总回报做更新，梯度估计方差很大，训练不稳定。课程把它概括成“trial and error”的正式版本，但试错并不等于随便试，而是要在回报噪声很大的条件下尽量学到正确方向。

3.3 Actor-Critic: 用 critic 降低方差

Actor-Critic 的思路是把策略更新和价值估计拆开:

- actor 负责输出动作策略;
- critic 负责估计价值函数。

一个批量版本的流程可以写成:

1. 从当前策略 $\pi_\theta(a | s)$ 采样状态和动作;
2. 用采样到的回报拟合一个价值函数 $\hat{V}_\phi(s)$;
3. 计算优势估计

$$\hat{A}_\pi(s_i, a_i) = r(s_i, a_i) + \gamma \hat{V}_\phi(s') - \hat{V}_\phi(s);$$

4. 用 $\hat{A}_\pi$ 加权策略梯度;
5. 按梯度更新 θ 。

这样做的直觉是: 与其直接用整条轨迹的总回报, 不如用一个学习到的 baseline 作为“参考线”, 只关心动作相对平均水平高了多少。

3.4 PPO: 限制每次更新的幅度

课程进一步指出, 策略梯度方法还有一个常见问题: policy collapse (策略坍塌)。也就是说, 训练曲线可能先很好, 随后突然急剧掉下去。PPO (Proximal Policy Optimization, 近端策略优化) 就是针对这个问题提出的。

PPO 的核心是 clipped surrogate objective。标准写法可以写成:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)},$$
$$L^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

其中:

- $r_t(\theta)$ 是当前策略相对于旧策略的概率比;
- $\hat{A}_t$ 是优势估计;
- ϵ 控制允许的更新幅度;
- clip 会把过大的变化截断。

PPO 的目的不是让策略每一步都走得最大, 而是让它稳步前进。对于机器人 RL 来说, 这一点尤其重要, 因为不稳定更新往往会直接转化成真实系统中的危险行为。

3.5 本章小结

REINFORCE 提供了最直接的 policy gradient 视角, Actor-Critic 通过 critic 减少方差, PPO 则进一步通过裁剪更新幅度来缓解策略坍塌。三者构成了这讲最核心的算法主线。

4 把 RL 用到机器人：现实中的优势与困难

4.1 为什么机器人是 RL 的自然应用场景

讲者指出，机器人非常适合被看成 agent，因为它本来就要在环境中感知、决策、执行，并通过反馈改进自己的行为。DQN 和 AlphaGo 的成功让人们更明确地看到：如果系统面对的是连续交互、长期回报和复杂状态，RL 可能比纯监督学习更自然。

4.2 真实世界 RL 的四个困难

课程把 real-world RL 的问题总结得很清楚：

- sample inefficient：真实世界采样太慢、太贵；
- unsafe：初始策略往往随机，可能损坏机器人或环境；
- reset 代价高：每一回合都要人工重置；
- reward definition 困难：真实任务中的奖励很难自动定义。

这四个困难解释了为什么“能在仿真里跑”几乎是机器人 RL 的前提，而不是附加能力。

真实世界 RL 的现实含义

很多“现实中训练出的成功系统”并不是在纯野外无条件自学习，而是背后有大量工程约束：安全边界、重置机制、收集流程、奖励设计和实验管理。

4.3 sim-to-real 作为折中路径

Sim-to-real 的逻辑是：先在 simulator 里把策略训好，再把同一策略部署到真实机器人上。这样做能够部分解决采样慢、成本高、重置难等问题，但它也引入了 sim-to-real gap，即仿真与真实之间的差异会导致策略性能下降。

课程在这里还提醒了一点：RL 训练曲线经常不像分类那样平滑。它可能突然掉点，同一组超参数换个随机种子就变样，不同代码库实现也会给出明显不同结果。换句话说，RL 的“不稳定”既来自算法本身，也来自训练系统的复杂性。

4.4 本章小结

机器人是 RL 的天然应用对象，但真实世界采样、安全、重置和奖励定义都使其难以直接落地。Sim-to-real 因而成为机器人 RL 的核心工程路线，同时也是整个领域最棘手的部分之一。

5 模拟器如何工作：从动力学到碰撞检测

5.1 为什么需要模拟器

讲者把 simulator 描述成机器人 RL 的基础设施。它的作用不只是“省钱”，更是给算法开发提供一个可重复、可并行、可调参的环境。课程中提到，先在 simulator 里开发算法，再把同样的算法训练到任务上，是当前实践中最常见的路径。

5.2 模拟循环的三部分

典型刚体模拟 (rigid body simulation) 循环包含三步：

1. forward dynamics: 根据当前状态和碰撞等信息求力、力矩、加速度；
2. numerical time integration: 把动力学量积分成下一步的速度和位置；
3. collision detection: 判断物体之间是否发生碰撞。

其中 collision detection 又可细分为 broad phase 和 narrow phase。前者先做粗筛，后者再做精确判定。课程点到的算法包括 Gilbert-Johnson-Keerthi (GJK) 和 Expanding Polytope Algorithm (EPA) 一类的窄相位方法。

5.3 牛顿与拉格朗日视角

讲者顺带回顾了两种常见动力学表达方式：

$$v_{t+\Delta t} = v_t + a\Delta t = v_t + \frac{F_{\text{ext}} + F_c}{m}\Delta t, \quad x_{t+\Delta t} = x_t + v_{t+\Delta t}\Delta t.$$

这是牛顿 (Newton) 视角：力直接决定加速度，再由加速度更新速度和位置。

另一种是拉格朗日 (Lagrange) 视角，课程写成

$$Q_i = \sum_j F_j \frac{dr_i}{dq_j}, \quad \frac{d}{dt} \frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = Q_i,$$

其中 $L = T - U$ 是拉格朗日量， T 是动能， U 是势能。这个视角的优势在于它更适合广义坐标，也更方便处理机器人系统中的复杂约束。

5.4 本章小结

模拟器的任务不是“画个动画”那么简单，而是把动力学、积分和碰撞检测连接起来，形成可训练的环境。对于机器人 RL 而言，模拟器本身就是算法的一部分。

6 总结与延伸

这一讲的主线可以压缩成一句话：RL 之所以适合机器人，是因为机器人任务天然顺序决策问题；RL 之所以难以直接落地，是因为真实世界采样昂贵、安全敏感且难以重置；而 simulator 和 sim-to-real 技巧，正是让 RL 真正走向机器人的关键支撑。

6.1 延伸阅读

- *Learning to Walk via Deep Reinforcement Learning*
- *Learning Dexterous In-Hand Manipulation*
- OpenAI Gym 相关文献，帮助理解标准化模拟接口如何影响 RL 研究生态

6.2 最后一句话

如果只记住一个结论，那就是：RL 不是单纯地学“动作标签”，而是在学一种会改变未来数据分布的行为策略；而在机器人里，这种学习几乎总要经过 simulator 与 sim-to-real 的双重考验。